

Лабораторная работа № 13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Студент: Чжао Синья

Учётная запись: zhaoxinya

Имя хоста: zhaoxinya

Номер студенческого билета: 113225832

Дисциплина: Архитектура компьютеров и операционные системы. Операционные системы

Содержание

1. Цель работы
 2. Задание работы
 3. Теоретические сведения
 4. Соглашения об именовании
 5. Ход выполнения работы
 6. Выводы
 7. Ответы на контрольные вопросы
 8. Библиография
-

Цель работы

Изучить основы программирования в оболочке ОС UNIX/Linux. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание работы

1. Используя команды `getopts` и `grep`, написать командный файл, анализирующий командную строку с ключами `-i` (входной файл), `-o` (выходной файл), `-p` (шаблон), `-c` (различать регистр), `-n` (нумерация строк), и выполняющий поиск строк по шаблону.

2. Написать на языке C программу, определяющую знак введённого числа, и завершающуюся функцией `exit(n)`. Командный файл должен вызвать программу и по коду возврата (`$?`) выдать сообщение.
3. Написать командный файл, создающий указанное число пронумерованных файлов `1.tmp`, `2.tmp`, ... `N.tmp`, и умеющий их удалять.
4. Написать командный файл, упаковывающий в архив все файлы указанной директории (с помощью `tar`). Модифицировать его так, чтобы паковывались только файлы, изменённые менее недели назад (использовать `find`).

Теоретические сведения

Команда `getopts`

`getopts` выполняет синтаксический анализ командной строки, выделяя опции и их аргументы. Синтаксис:

```
getopts option-string variable [arg ...]
```

Если после буквы опции стоит двоеточие, значит у этой опции есть аргумент, который помещается в переменную `OPTARG`. Принято использовать `getopts` в цикле `while` с оператором `case`:

```
while getopts "i:o:p:Cn" opt; do
  case $opt in
    i) infile=$OPTARG;;
    o) outfile=$OPTARG;;
    p) pattern=$OPTARG;;
    C) case_sensitive=1;;
    n) show_line_numbers=1;;
  esac
done
```

Команда `grep`

```
grep [-опции] шаблон файл
```

Основные опции:

- `-i` — игнорировать регистр;
- `-n` — выдавать номера строк;

- `-C` — выводить контекст;
- `-v` — инвертировать совпадение.

Специальные переменные

- `$?` — код завершения последней команды;
- `$#` — число аргументов;
- `$1` , `$2` , ... — позиционные параметры.

Функция `exit` в C

`exit(n)` — завершение программы с кодом `n` , который становится доступен в shell как `$?` .

Управляющие конструкции `bash`

- `if ... then ... elif ... else ... fi`
- `case ... esac`
- `for ... do ... done`
- `while ... do ... done`
- `until ... do ... done`

Команды `find` и `tar`

```
find DIR -type f -mtime -7 -print0
```

- `-type f` — только файлы;
- `-mtime -7` — изменённые менее 7 дней назад;
- `-print0` — разделять имена нулевым байтом (для `tar --null -T -`).

```
tar -czf archive.tar.gz --null -T -
```

- `-c` — создать архив;
- `-z` — сжать gzip;
- `-f` — имя архива;
- `--null -T -` — читать список файлов со стандартного ввода, разделённых `\0` .

Соглашения об именовании

- Пользователь ВМ: `zhaoxinya`
- Имя хоста: `zhaoxinya`
- Имя виртуальной машины: `ZhaoXinya2`

Проверка:

```
whoami  
hostname  
id
```

```
zhaoxinya@zhaoxinya:~$ whoami  
zhaoxinya  
zhaoxinya@zhaoxinya:~$ hostname  
zhaoxinya  
zhaoxinya@zhaoxinya:~$ id  
uid=1000(zhaoxinya) gid=1000(zhaoxinya) groups=1000(zhaoxinya),10(wheel) context  
=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Ход выполнения работы

Задание 1. Командный файл с `getopts` и `grep`

Создан файл `~/mygrep.sh`:

```
nano ~/mygrep.sh
```

Исходный код:

```
#!/bin/bash
while getopts "i:o:p:Cn" opt; do
case $opt in
i) INFILE="$OPTARG";;
o) OUTFILE="$OPTARG";;
p) PATTERN="$OPTARG";;
C) CASE="-C";;
n) NUMS="-n";;
esac
done
if [ -z "$INFILE" ] || [ -z "$PATTERN" ]; then
echo "Usage: $0 -i input -p pattern [-o output] [-C] [-n]"
exit 1
fi
if [ -z "$OUTFILE" ]; then
grep $CASE $NUMS "$PATTERN" "$INFILE"
else
grep $CASE $NUMS "$PATTERN" "$INFILE" > "$OUTFILE"
echo "Result written to $OUTFILE"
fi
```

GNU nano 8.3
/home/zhaoxinya/mygrep.sh
Modified

1 #!/bin/bash
2 while getopts "i:o:p:Cn" opt; do
3 case \$opt in
4 i) INFILE="\$OPTARG";;
5 o) OUTFILE="\$OPTARG";;
6 p) PATTERN="\$OPTARG";;
7 C) CASE="-C";;
8 n) NUMS="-n";;
9 esac
10 done
11 if [-z "\$INFILE"] || [-z "\$PATTERN"]; then
12 echo "Usage: \$0 -i input -p pattern [-o output] [-C] [-n]"
13 exit 1
14 fi
15 if [-z "\$OUTFILE"]; then

^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^_ Replace
^U Paste
^J Justify

```

16 grep $CASE $NUMS "$PATTERN" "$INFILE"
17 else
18 grep $CASE $NUMS "$PATTERN" "$INFILE" > "$OUTFILE"
19 echo "Result written to $OUTFILE"
20 fi
21

```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

Установлены права и выполнен запуск:

```

chmod +x ~/mygrep.sh
~/mygrep.sh -i /etc/passwd -p root -n

```

Результат:

```

1:root:x:0:0:Super User:/root:/bin/bash
10:operator:x:11:0:operator:/root:/usr/sbin/nologin

```

Первоначально возникало предупреждение `line 11: [: missing `']`, связанное с отсутствием пробела между `]` и `;`. После корректной записи конструкции `[...] || [...]` предупреждение исчезло.

```

zhaoxinya@zhaoxinya:~$ nano ~/mygrep.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/mygrep.sh
zhaoxinya@zhaoxinya:~$ ~/mygrep.sh -i /etc/passwd -p root -n
/home/zhaoxinya/mygrep.sh: line 11: [: missing `']
1:root:x:0:0:Super User:/root:/bin/bash
10:operator:x:11:0:operator:/root:/usr/sbin/nologin

```

Задание 2. Программа на С и анализ кода возврата

Создан файл `~/checknum.c` :

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int n;
printf("Enter number: ");
scanf("%d", &n);
if (n > 0) exit(1);
else if (n < 0) exit(2);
else exit(0);
}
```

```
GNU nano 8.3 /home/zhaoxinya/checknum.c Modified
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main() {
4 int n;
5 printf("Enter number: ");
6 scanf("%d", &n);
7 if (n > 0) exit(1);
8 else if (n < 0) exit(2);
9 else exit(0);
10 }
11
```

^G Help **^O** Write Out **^F** Where Is **^K** Cut **^T** Execute
^X Exit **^R** Read File **^** Replace **^U** Paste **^J** Justify

Компиляция:

```
gcc ~/checknum.c -o ~/checknum
```

```
zhaoxinya@zhaoxinya:~$ nano ~/checknum.c
zhaoxinya@zhaoxinya:~$ gcc ~/checknum.c -o ~/checknum
```

Создан командный файл `~/checknum.sh` :

```
#!/bin/bash
~/checknum
R=$?
case $R in
0) echo "Number is zero";;
1) echo "Number is positive";;
2) echo "Number is negative";;
*) echo "Unknown exit code: $R";;
esac
```

```
GNU nano 8.3 /home/zhaoxinya/checkum.sh Modified
1 #!/bin/bash
2 ~/checknum
3 R=$?
4 case $R in
5 0)echo "Number is zero";;
6 1)echo "Number is positive";;
7 2)echo "Number is negative";;
8 *)echo "Unkonwn exit code: $R";;
9 esac
10
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

Установлены права и выполнены три проверки:

```
chmod +x ~/checknum.sh
~/checknum.sh
```

```
zhaoxinya@zhaoxinya:~$ chmod +x ~/checknum.sh
zhaoxinya@zhaoxinya:~$ ~/checknum.sh
Enter number: 5
Number is positive
zhaoxinya@zhaoxinya:~$ ~/checknum.sh
Enter number: -3
Number is negative
zhaoxinya@zhaoxinya:~$ ~/checknum.sh
Enter number: 0
Number is zero
zhaoxinya@zhaoxinya:~$
```

Результаты:

```
Enter number: 5
Number is positive

Enter number: -3
Number is negative

Enter number: 0
Number is zero
```

Программа на C возвращает через `exit(n)` код, который в командном файле анализируется с помощью `$?` и оператора `case`.

Задание 3. Создание и удаление N файлов

Создан файл `~/mkfiles.sh`:

```
#!/bin/bash
if [ $# -ne 2 ]; then
echo "Usage: $0 <count> <create|delete>"
exit 1
fi
N=$1
ACTION=$2
if [ "$ACTION" = "create" ]; then
for i in $(seq 1 $N); do
touch "$i.tmp"
done
echo "Created $N files"
elif [ "$ACTION" = "delete" ]; then
for i in $(seq 1 $N); do
rm -f "$i.tmp"
done
echo "Deleted $N files"
else
echo "Unknown action: $ACTION"
exit 1
fi
```

GNU nano 8.3 /home/zhaoxinya/mkfiles.sh Modified

```
#!/bin/bash
if [ $# -ne 2 ]; then
echo "Usage: $0 <count> <create|delete>"
exit 1
fi
N=$1
ACTION=$2
if [ "$ACTION" = "create" ]; then
for i in $(seq 1 $N); do
touch "$i.tmp"
done
echo "Created $N files"
elif [ "$ACTION" = "delete" ]; then
for i in $(seq 1 $N); do
rm -f "$i.tmp"
```

^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^X Exit
^R Read File
^_ Replace
^U Paste
^J Justify

```
done
echo "Deleted $N files"
else
echo "Unknown action: $ACTION"
exit 1
fi

```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

Установлены права и выполнены проверки:

```
chmod +x ~/mkfiles.sh
~/mkfiles.sh 5 create
ls *.tmp
~/mkfiles.sh 5 delete
ls *.tmp

```

Результат:

```
Created 5 files
1.tmp 2.tmp 3.tmp 4.tmp 5.tmp
Deleted 5 files
ls: cannot access '*.tmp': No such file or directory

```

Скрипт корректно создаёт и удаляет заданное число файлов с именами `1.tmp ... N.tmp`.

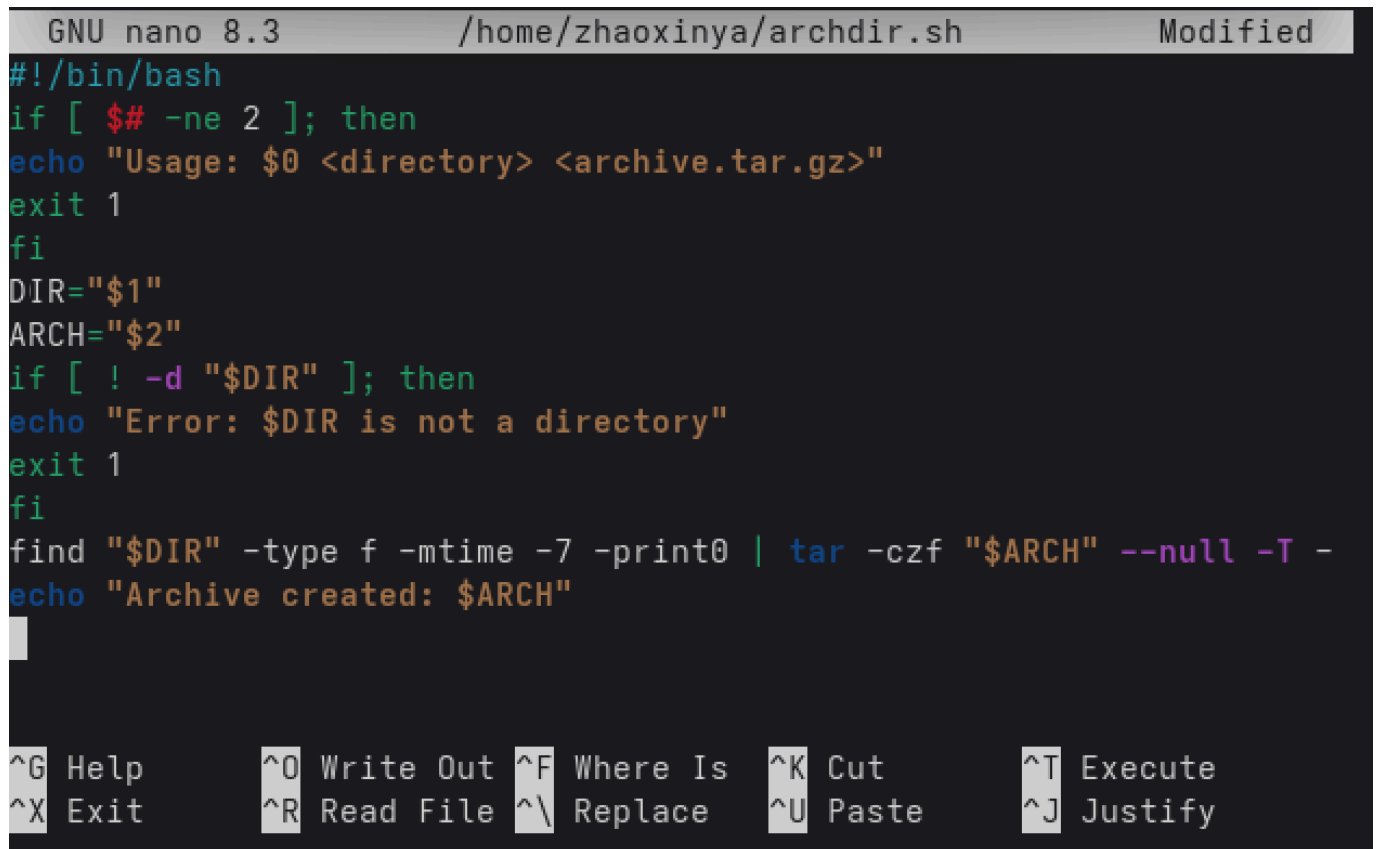
```
zhaoxinya@zhaoxinya:~$ nano ~/mkfiles.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/mkfiles.sh
zhaoxinya@zhaoxinya:~$ ~/mkfiles.sh 5 create
Created 5 files
zhaoxinya@zhaoxinya:~$ ls *.tmp
1.tmp 2.tmp 3.tmp 4.tmp 5.tmp
zhaoxinya@zhaoxinya:~$ ~/mkfiles.sh 5 delete
Deleted 5 files
zhaoxinya@zhaoxinya:~$ ls *.tmp
ls: cannot access '*.tmp': No such file or directory
zhaoxinya@zhaoxinya:~$

```

Задание 4. Упаковка файлов с tar и find

Создан файл `~/archdir.sh` :

```
#!/bin/bash
if [ $# -ne 2 ]; then
echo "Usage: $0 <directory> <archive.tar.gz>"
exit 1
fi
DIR="$1"
ARCH="$2"
if [ ! -d "$DIR" ]; then
echo "Error: $DIR is not a directory"
exit 1
fi
find "$DIR" -type f -mtime -7 -print0 | tar -czf "$ARCH" --null -T -
echo "Archive created: $ARCH"
```



```
GNU nano 8.3 /home/zhaoxinya/archdir.sh Modified
#!/bin/bash
if [ $# -ne 2 ]; then
echo "Usage: $0 <directory> <archive.tar.gz>"
exit 1
fi
DIR="$1"
ARCH="$2"
if [ ! -d "$DIR" ]; then
echo "Error: $DIR is not a directory"
exit 1
fi
find "$DIR" -type f -mtime -7 -print0 | tar -czf "$ARCH" --null -T -
echo "Archive created: $ARCH"
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Установлены права и выполнен запуск:

```
chmod +x ~/archdir.sh
~/archdir.sh ~ /tmp/mybackup.tar.gz
ls -l /tmp/mybackup.tar.gz
tar -tzf /tmp/mybackup.tar.gz | head
```

Результат:

```
tar: Removing leading `/' from member names
tar: Removing leading `/' from hard link targets
tar: /home/zhaoxinya/ski.plases/feathers: Cannot open: Permission denied
tar: Exiting with failure status due to previous errors
Archive created: /tmp/mybackup.tar.gz
-rw-r--r--. 1 zhaoxinya zhaoxinya 540454939 Oct 8 16:32 /tmp/mybackup.tar.gz
home/zhaoxinya/.mozilla/firefox/...
...
```

Сообщение `Removing leading '/' from member names` — стандартное поведение `tar`.

Сообщение `Cannot open: Permission denied` относится к файлу `ski.plases/feathers`, у которого в лабораторной работе № 7 намеренно было снято право чтения, поэтому `tar` не смог его прочитать и пропустил. Архив создан успешно, что подтверждается размером файла (~515 МБ) и успешным выводом оглавления через `tar -tzf`.

```
zhaoxinya@zhaoxinya:~$ nano ~/archdir.sh
zhaoxinya@zhaoxinya:~$ chmod +x ~/archdir.sh
zhaoxinya@zhaoxinya:~$ ~/archdir.sh ~ ~/mybackup.tar.gz
tar: Removing leading `/' from member names
tar: Removing leading `/' from hard link targets
tar: /home/zhaoxinya/ski.plases/feathers: Cannot open: Permission denied
tar: /home/zhaoxinya/mybackup.tar.gz: file changed as we read it
tar: Exiting with failure status due to previous errors
Archive created: /home/zhaoxinya/mybackup.tar.gz
zhaoxinya@zhaoxinya:~$ ls -l ~/mybackup.tar.gz
-rw-r--r--. 1 zhaoxinya zhaoxinya 360521916 Oct 8 16:28 /home/zhaoxinya/mybackup.tar.gz
zhaoxinya@zhaoxinya:~$ tar -tzf ~/mybackup.tar.gz | head
home/zhaoxinya/.mozilla/firefox/Crash Reports/InstallTime20250325084202
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/times.json
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/.parentlock
```

```
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/crashes/stor
e.json.mozlz4
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/compatibilit
y.ini
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/cookies.sqli
te
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/storage.sqli
te
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/storage/ls-a
rchive.sqlite
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/storage/perm
anent/chrome/.metadata-v2
home/zhaoxinya/.mozilla/firefox/1qaw1b5w.default-release/storage/perm
anent/chrome/idb/3870112724rsegmnoittet-es.sqlite
```

```
zhaoxinya@zhaoxinya:~$ ~/archdir.sh ~ /tmp/mybackup.tar.gz
tar: Removing leading '/' from member names
tar: Removing leading '/' from hard link targets
tar: /home/zhaoxinya/ski.plases/feathers: Cannot open: Permission den
ied
tar: Exiting with failure status due to previous errors
Archive created: /tmp/mybackup.tar.gz
zhaoxinya@zhaoxinya:~$ ls -l /tmp/mybackup.tar.gz
-rw-r--r--. 1 zhaoxinya zhaoxinya 540454939 Oct  8 16:32 /tmp/mybacku
p.tar.gz
```

Выводы

В ходе лабораторной работы были изучены:

- команда `getopts` для синтаксического анализа командной строки;
- взаимодействие командного файла с программой на языке C через код возврата `exit(n)` и переменную `$?`;
- использование циклов `for`, конструкций `if/elif/else` и `case` в командных файлах;
- работа с командами `find`, `tar` для архивации файлов по заданному критерию.

Написаны и отлажены 4 файла: `mygrep.sh`, `checknum.c` + `checknum.sh`, `mkfiles.sh`, `archdir.sh`. Все скрипты проверены на реальных данных.

Ответы на контрольные вопросы

1. Каково предназначение команды `getopts`?

`getopts` служит для разбора опций (ключей) командной строки в командных файлах. Она выделяет буквенные опции, а при наличии двоеточия в строке опций — и их аргументы (доступны через `$OPTARG`). Обычно используется в цикле `while` вместе с `case`.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (`*`, `?`, `[ ]`) используются оболочкой для генерации имён файлов по шаблону. Например, `*.c` подставляет все файлы с расширением `.c`, `[a-z]*` — все файлы, начинающиеся со строчной латинской буквы. Перед передачей команде shell сам раскрывает эти шаблоны.

3. Какие операторы управления действиями вы знаете?

- `if ... then ... fi`
- `if ... then ... elif ... else ... fi`
- `case ... esac`
- Логические операторы `&&` и `||`
- Операторы сравнения `-eq`, `-ne`, `-lt`, `-gt`, `-le`, `-ge`

4. Какие операторы используются для прерывания цикла?

- `break` — полностью выходит из цикла;
- `continue` — прерывает текущую итерацию и переходит к следующей.

5. Для чего нужны команды `false` и `true`?

- `true` — всегда возвращает код завершения 0 (успех);
- `false` — всегда возвращает ненулевой код (неудача).

Они используются для организации бесконечных циклов (`while true; do ... done`) и в условиях, где нужен явно заданный код возврата.

6. Что означает строка `if test -f man$s/$i.$s`, встречаемая в командном файле?

Проверяется, существует ли обычный файл (`-f`), путь к которому склеен из значений переменных:

- `man$s` — каталог `man` + суффикс `$s`;
- `$i` — имя файла (переменная);
- `.$s` — расширение `$s`.

То есть условно: если существует файл `manN/число.N`, то выполнить соответствующую ветку.

7. Объясните разницу между конструкциями `while` и `until`.

- `while условие; do ... done` — цикл выполняется, пока условие **истинно**;
- `until условие; do ... done` — цикл выполняется, пока условие **ложно** (эквивалентно `while not условие`).

Обе конструкции проверяют условие перед каждой итерацией, но инвертируют логику проверки.

Библиография

1. Курячий Г. В., Маслинский К. А. Операционная система Linux. — М.: Интуит.Ру, 2005. — 392 с.
2. Робачевский А. Операционная система UNIX. — СПб.: БХВ-Петербург, 2010. — 656 сс.
3. Robbins A. Bash Pocket Reference. — O'Reilly Media, 2016. — 156 сс.